Small Errors, Big Consequences: Lessons from Data Mismanagement in Business

Karla Hernández 400916125

Fresenius University of Applied Science

Data Science and Data Analytics (WS 2025/26)

Prof. Dr. Stephan Huber

2025-02-17

Author Note

Correspondence concerning this article should be addressed to Karla Hernández 400916125, Email: hernandez.karla@stud.hs-fresenius.de

Abstract

Data plays a major role in business decisions, from production and hiring to marketing and forecasting. However, even tiny mistakes in a dataset can lead to completely wrong conclusions and very expensive consequences; so if the data has mistakes, the decisions will have mistakes. The challenge is that many data errors are small and easy to overlook, especially when working in RStudio, because R does not warn us when a result is logically wrong; it only warns us when the code fails. This handout explains how small issues such as missing values, duplicated rows, incorrect data types or poorly executed joins can quietly break calculations and models. Using simple examples of bad practice vs better practice in R, you'll learn how common habits can hide errors and how better habits prevent them. The goal is not about perfection, but about developing good routines that help us catch mistakes early, before they affect real business decisions.

Small Errors, Big Consequences: Lessons from Data Mismanagement in Business

Word count: 1022

1 "Small" Erros

When people think about data problems in business, they often imagine something dramatic; like a system failure, a cyberattack, or a huge coding bug. But in reality, business data disasters usually don't come from big dramatic mistakes. They come from very small, very ordinary mistakes that slip through because nobody notices them.

Here's why:

- Companies collect tons of data every day.
- Teams are under pressure to work fast and deliver results.
- Everyone assumes that software will catch mistakes automatically.
- RStudio runs perfectly even if the data is wrong.

So a tiny mistake, like a missing value or a duplicated customer, might not look dangerous at first. But once that mistake is used in reports, dashboards, forecasts, or decisions… it suddenly becomes a big problem with big consequences.

2 What May Count as a "Small Error"?

Here are some examples of small mistakes that can break results later:

Table 1

What may be considered as "small error" examples

Error	What it Means
"Mexico" vs "México"	R treats them as two different categories
Number stored as text	Math doesn't work the way we expect
Missing values	Averages and totals become wrong
Duplicate rows	Customers or employees counted twice
Wrong join	Rows multiply or disappear silently

3 Tiny Erros, Big Consequences

The previous mistakes look tiny, but later they affect totals, predictions, and business decisions. Let's take now a look of how these errors can be related to real life examples:

Table 2

Examples in real life

Area	Tiny Error	Big Result
Retail	Missing value in sales	Too little production that leads to lost on sales
HR	Employee duplicated during merge	Over-hired people that leads to unnecessary cost
Marketing	Duplicated customers	Overspent on ads

4 Bad Practice vs Better Practice in R

Now, we will see some examples on how to avoid small mistakes in R and replace risky habits with safer, clearer, and more reliable practices. Many of the "bad practices" shown below are considered wrong mainly because even though R runs them perfectly fine, the results are quietly broken. This means we will get broken results used in reports, dashboards, or business decisions, and here is when the damage is done.

This section is about learning to recognize moments where we *assume* something instead of **checking**, we let R *"guess"* instead of giving **clear instructions**, we work *faster* instead of **safer**, and we *trust* the output simply because **"there was no error message"**.

By comparing bad practices vs better practices, we can clearly see how very small choices in code, tiny details, can be the difference between a *result that is silently wrong* and a **result that can be trusted.**

5 Example of a Bad Practice

```
# BAD import: let R guess names and types
data_bad <- read.csv("shopping_behavior.csv")

# BAD filter (forgetting to handle NA, using base [] correctly this time but no clean
young_bad <- data_bad[data_bad$Age < 30, ]

# BAD join using merge() with the same location_info
location_info <- data.frame(
  Location = c("California", "New York", "Texas", "Florida"),
  Region   = c("West", "East", "South", "South"),
  stringsAsFactors = FALSE
)
```

```
# Rename columns to match good version later
names(result_bad) <- c("Location_clean", "avg_spend_bad")
result_bad$avg_spend_bad <- round(result_bad$avg_spend_bad, 2)

result_bad
print(result_bad, n = Inf)
View(result_bad)
```

6 Example of a Good Practice

```
library(dplyr)
library(lubridate)

# 1) Read data with good practices
data_good <- read.csv(
  "shopping_behavior.csv",
  stringsAsFactors = FALSE,
  check.names = FALSE   # keeps "Purchase Amount (USD)" as-is
)

# 2) Parse dates correctly
data_good <- data_good %>%
  mutate(
    Date_good = dmy(Date)
  )

# 3) Remove exact duplicate rows (we know we injected some)
data_nodup <- data_good %>%
  distinct(.keep_all = TRUE)

# 4) Keep only customers under 30
young_good <- data_nodup %>%
```

```r
  filter(Age < 30)

# 5) Location lookup table (regions are just for context)
location_info <- data.frame(
  Location = c("California", "New York", "Texas", "Florida"),
  Region   = c("West", "East", "South", "South"),
  stringsAsFactors = FALSE
)

# 6) Safe join: keep all young customers, add Region if Location matches
merged_good <- young_good %>%
  left_join(location_info, by = "Location")

# 7) Clean up Location variants into one standard name
final_clean <- merged_good %>%
  mutate(
    Location_clean = case_when(
      Location %in% c("California", "california", "Californi", "CA") ~ "California",
      Location %in% c("New York", "NewYork", "N. York")              ~ "New York",
      TRUE ~ Location
    )
  ) %>%
  group_by(Location_clean) %>%
  summarise(
    avg_spend_young = mean(`Purchase Amount (USD)`, na.rm = TRUE),
    n_customers     = n(),
    .groups = "drop"
  ) %>%
  arrange(Location_clean)

# 8) Round numbers to make the table nice for printing
final_clean$avg_spend_young <- round(final_clean$avg_spend_young, 2)
```

```
# 9) Print final result nicely
final_clean


print(final_clean, n = Inf)
View(final_clean)
```

7 Lessons from Data Mismanagement in Business

1. A tiny mistake in data can cause a huge mistake in a business decision.
2. RStudio won't warn you when the result is wrong, only when the code is.
3. Never assume the data is clean, always check it.
4. Good analysis starts before modeling with data validation.
5. Being explicit in your code prevents disasters.
6. Documentation and reproducible scripts protect you from mistakes.
7. Joins are one of the most dangerous operations, check them twice.
8. Data types matter more than most people realize.
9. Peer review is essential.
10. Great analysts aren't those who make no mistakes, they are the ones who catch mistakes early.

Table 3

Best common practices

Task	Bad Practice	Better Practice	Explanation
Importing	`read.csv("file.csv")`	`read.csv("file.csv", stringsAsFactors = FALSE)`	R doesn't convert text into categories automatically.
Missing values	`mean(x)`	`mean(x, na.rm = TRUE)`	Get real averages instead of NA results that look "valid" but are unusable.
Joining tables	`merge(df1, df2)`	`left_join(df1, df2, by = "ID") + anti_join()`	Avoid accidental duplicates and can see which rows didn't match.
Data types	Assuming R knows the type	`str()`	Verify and enforce data types instead of hiping R guessed correctly. ***Only applicable for versions prior to R 4.0***
Checking data	Not looking at the data	`summary()`, `head()`, `table()`, `view()`	Catch mistakes before modeling instead of discovering them too late.
Filtering	`data[data$age > 18]`	`filter(data, age > 18)`	Keeps the dataset clean and consistent instead of returning unpredictable formats.
Removing duplicates	`unique(data)`	`distinct(data, .keep_all = TRUE)`	Removes duplicates but also lets us see exactly which rows were duplicated.
Renaming columns	`colnames(data)[3] <- "sales"`	`rename(data, sales = old_name)`	Rename the correct column even if column order later changes.
Selecting columns	`data[, c(1,2,5]`	`select(data, name, price, region)`	Safer because column names don't change, column numbers do.
Dates	`as.Date(Date)`	`as.Date(date, format="%d/%m/%Y")` or `lubridate::dmy()`	Prevents R from guessing the wrong date format, which happens often.

8 Affidavit

I hereby affirm that this submitted paper was authored unaided and solely by me. Additionally, no other sources than those in the reference list were used. Parts of this paper, including tables and figures, that have been taken either verbatim or analogously from other works have in each case been properly cited with regard to their origin and authorship. This paper either in parts or in its entirety, be it in the same or similar form, has not been submitted to any other examination board and has not been published.

I acknowledge that the university may use plagiarism detection software to check my thesis. I agree to cooperate with any investigation of suspected plagiarism and to provide any additional information or evidence requested by the university.

Checklist:

- ☒ The handout contains 3-5 pages of text.
- ☒ The submission contains the Quarto file of the handout.
- ☒ The submission contains the Quarto file of the presentation.
- ☒ The submission contains the HTML file of the handout.
- ☒ The submission contains the HTML file of the presentation.
- ☒ The submission contains the PDF file of the handout.
- ☒ The submission contains the PDF file of the presentation.
- ☒ The title page of the presentation and the handout contain personal details (name, email, matriculation number).
- ☒ The handout contains a bibliography, created using BibTeX with an APA citation style.
- ☒ Either the handout or the presentation contains R code that proofs the expertise in coding.
- ☒ The filled out Affidavit.
- ☒ The link to the presentation and the handout published on GitHub.
- ☐ In group work, each student's contribution is clearly defined, and individual performance can be assessed using specified sections, page numbers, or other objective criteria.

[Karla Hernández,] [2025-02-17,] [Köln, Germany]